

Resumen: ¿Por qué Johnny no puede escribir prompts?

Cómo los no-expertos en IA intentan (y fallan) al diseñar prompts para modelos de lenguaje (LLMs)

Zamfirescu-Pereira, Wong, Hartmann y Yang · CHI '23

1. Introducción y contexto

Los modelos grandes de lenguaje (LLMs) como GPT-3 o ChatGPT pueden mantener conversaciones fluidas y realizar tareas variadas sin necesidad de entrenamiento adicional. Basta con anteponer un **prompt** (una instrucción en lenguaje natural) para orientar sus respuestas. Esto abre la puerta a que personas **sin conocimientos técnicos de IA ni de programación** puedan crear aplicaciones conversacionales.

Sin embargo, aunque escribir prompts *parece* sencillo, diseñar prompts efectivos y robustos es difícil incluso para expertos. Este trabajo investiga cómo abordan intuitivamente el diseño de prompts los usuarios no expertos y qué dificultades enfrentan, con el fin de orientar el diseño de futuras herramientas y la educación sobre prompts.

2. Conceptos clave para entender el estudio

2.1 ¿Qué es un prompt y qué es "prompt engineering"?

Un **prompt** es el texto (instrucciones, ejemplos, contexto) que se le da al modelo antes de que genere una respuesta. El **prompt engineering** consiste en: (a) identificar los contextos en los que el modelo falla, (b) idear estrategias de prompt para corregir esos fallos, y (c) evaluar sistemáticamente si esas estrategias funcionan de manera general.

2.2 El paradigma "pre-entrenar, promptear, predecir"

En el enfoque tradicional de chatbots, los diseñadores debían: definir la funcionalidad, diseñar un flujo de diálogo, entrenar modelos supervisados con datos etiquetados e iterar. El nuevo paradigma con LLMs permite crear un chatbot funcional **solo escribiendo prompts**, sin código ni datos de entrenamiento.

2.3 Estrategias de prompt conocidas por ser efectivas

Investigaciones previas sugieren tres estrategias útiles:

- **Dar ejemplos** de las interacciones deseadas dentro del propio prompt.
- **Escribir prompts que parezcan código** (usando plantillas y separadores claros entre instrucción y datos).
- **Repetirse**: reiterar la misma idea con variaciones puede mejorar la salida.

2.4 Barreras conocidas en programación para no expertos

El estudio se apoya en barreras ya identificadas en sistemas de programación para usuarios finales (EUPS) y en aprendizaje automático interactivo (iML):

- **Diseño:** "no sé qué quiero que haga la computadora".
- **Selección:** "sé qué quiero, pero no sé qué usar".
- **Coordinación:** "sé qué usar, pero no cómo combinarlo".
- **Uso:** "sé qué usar, pero no cómo".
- **Comprensión:** "creí saber usarlo, pero no hizo lo esperado".
- **Información:** "sé por qué falló, pero no cómo verificarlo".

3. Método: la herramienta BotDesigner

Los autores construyeron **BotDesigner**, una herramienta web sin código que sirve como "sonda de diseño" (design probe). Permite: (1) crear un chatbot únicamente escribiendo prompts, y (2) evaluar sistemáticamente el efecto de los cambios en los prompts.

3.1 Componentes del prompt en BotDesigner

- **Preamble (preámbulo)**: instrucciones en lenguaje natural para el bot (rol, tarea, pasos, ejemplos).
- **First Turns (primeros turnos)**: mensajes iniciales fijos con los que arranca cada conversación.
- **Reminder (recordatorio)**: texto que se inserta justo antes de cada respuesta del bot.

3.2 Dos vistas de la herramienta

Vista de Conversación: el diseñador chatea con el bot, etiqueta errores y prueba cambios de prompt localmente (con botones "retry" y "edit").

Error Browser: permite ejecutar un prompt modificado sobre todas las conversaciones guardadas para ver si los errores previos se corrigieron o si aparecieron nuevos.

3.3 Flujo de trabajo propuesto (7 pasos)

1. Definir un prompt base ("baseline").
2. Generar conversaciones de ejemplo para ver qué hace el bot.
3. Identificar y etiquetar errores.
4. Elegir un error concreto para corregir.
5. Modificar el prompt y probarlo localmente (retry).
6. Probar el cambio globalmente en todas las conversaciones.
7. Iterar sobre nuevos errores.

3.4 Participantes y tarea

Se reclutaron **10 participantes** sin experiencia real en diseño de prompts (profesores, estudiantes de posgrado, diseñadores, ingenieros, etc.). La tarea consistía en mejorar un chatbot base para que imitara a la chef Carla Lalli del programa *Back to Back Chef*: hacer bromas, dar analogías, simplificar conceptos, confirmar pasos y conversar socialmente mientras guiaba al usuario a cocinar una tempura de vegetales.

Los participantes disponían de hasta 1 hora, se les pedía **pensar en voz alta** y podían explorar libremente. Los investigadores intervenían solo si el usuario se quedaba varado.

4. Hallazgos principales

Los resultados se resumen en dos ideas centrales:

- Todos los participantes lograron iterar sobre los prompts de forma **oportunista** (prueba y error puntual), pero **ninguno** avanzó de manera sistemática o robusta.
- Las principales dificultades vienen de: (a) **sobre-generalizar** a partir de un solo éxito o fracaso, y (b) tratar al bot como si fuera un humano, arrastrando expectativas sociales al escribir instrucciones.

4.1 Cómo abordaron el diseño de prompts (flujo típico)

1. Empiezan una conversación con el bot base hasta encontrar un error (falta de humor, listas demasiado largas, no confirmar pasos, explicaciones raras).
2. Suelen detenerse ante el primer error, sin explorar más contextos.
3. Modifican el preámbulo con instrucciones explícitas (p. ej., "haz chistes" o "no digas X").
4. Cuando se atascan, piden ayuda al investigador o buscan en Google.
5. Consideran resuelto un problema tras **una sola** respuesta correcta.
6. Continúan o inician otra conversación hasta encontrar el próximo error.

4.2 Dificultades al generar prompts

- **Al empezar:** la mitad no sabía qué comportamientos podía pedir ("¿qué puedo hacer aquí?"). Confusión sobre su propio rol (¿soy Carla o soy el productor?).
- **Elegir la instrucción correcta:** pequeños fracasos los llevaban a concluir que el bot era *incapaz* de la tarea, sin probar otras redacciones.
- **Expectativas humanas:** mezclaban órdenes dirigidas al bot con órdenes dirigidas al usuario, asumiendo que el bot distinguiría. También creían que el bot recordaba conversaciones anteriores.
- **Cortesía social:** muchos usaban "por favor" y un tono neutro incluso frustrados. Rara vez repetían frases (aunque la repetición ayuda).
- **Preferencia por instrucciones directas sobre ejemplos:** casi nadie copió diálogos de ejemplo al preámbulo por iniciativa propia; los pocos que lo hicieron por sugerencia lo consideraban "trampa", pese a que era muy efectivo.
- **Buscar ayuda:** quienes buscaron en la web no supieron cómo trasladar ejemplos de otros dominios a su caso.

4.3 Dificultades al evaluar prompts

- **Recolección de datos insuficiente:** nadie generó más de 1-2 conversaciones antes de intentar corregir.
- **Poco uso del etiquetado de errores:** al ver que las etiquetas no las usaba el modelo, dejaron de ponerlas.

- **Cero uso de la prueba sistemática:** se conformaban con una única respuesta "correcta" tras el cambio. Este exceso de confianza aparece incluso en programadores profesionales.
- Algunos habrían preferido una vista que **comparara lado a lado** plantillas y sus efectos palabra por palabra.

4.4 Dificultades al explicar los efectos del prompt

- **Generalización entre contextos:** un prompt que funcionaba en un caso fallaba en otro similar, generando confusión.
- **Atribuir todo a incapacidad del modelo:** si un "No digas X" no funcionaba, asumían que el bot no podía obedecer, en vez de reformular. La literatura indica que decir "haz Y" suele ser más eficaz que "no hagas X", pero los usuarios usan mucho más la forma negativa.
- **Ejemplo ilustrativo:** P4 pasó minutos intentando bloquear una frase con "Don't say...". Solo funcionó cuando eliminó el "Don't" y expresó la instrucción en positivo.
- Todos los participantes preguntaron en algún momento "¿por qué hizo eso?" y los investigadores tuvieron que responder "no lo sé": los LLMs siguen siendo cajas negras.

5. Discusión: implicaciones

5.1 Para la formación y educación de usuarios

- Enseñar a **recolectar más datos** (más conversaciones) antes de dar por resuelto un problema.
- Crear **repositorios de ejemplos** (como el *DALL·E 2 Prompt Book*) con prompts que funcionan y en qué contextos.
- Explicar que estrategias como **usar ejemplos y repetir** son efectivas, aunque "se sientan raras".
- Aclarar el **ciclo de vida** de las instrucciones: el preámbulo persiste, la conversación no.
- Fomentar explícitamente la **evaluación sistemática**, no solo la prueba puntual.

5.2 Para el diseño de futuras herramientas

- Ofrecer **heurísticas de robustez**: cuántas conversaciones y cuántos ejemplos por error son suficientes.
- Hacer **descubribles** los buenos ejemplos justo cuando el usuario los necesita.
- Mostrar visualmente la relación **causa-efecto** entre cambios en el prompt y cambios en las respuestas.
- Mitigar el efecto "la computadora es un actor social" (CASA, Nass): permitir que el usuario piense que instruye a un tercero, usar plantillas al estilo Mad Libs, o repetir instrucciones "bajo el capó".
- Considerar diseño **"con costuras visibles"** (seamful design): mostrar que el sistema no es humano puede ayudar a interactuar mejor con él.

5.3 Limitaciones del estudio

La muestra (10 personas, mayoría académicos STEM) no representa a la población general; el tiempo de la tarea fue limitado; se usó solo un dominio (chatbots instructivos). Además, los autores reconocen implicaciones sociales más amplias de los LLMs (derechos de autor, acoso, automatización de empleos) que quedan fuera del alcance.

5.4 Preguntas abiertas para futura investigación

- ¿Estos comportamientos aparecen en poblaciones más diversas y en dominios no conversacionales?
- ¿Cuánto influyen realmente las expectativas sociales? ¿Bastan los ejemplos para contrarrestarlas?
- ¿Existen diferencias culturales (p. ej., culturas de alto contexto) en estos sesgos?
- ¿Cómo ajustar las expectativas de los usuarios sobre las **capacidades reales** de los LLMs (que no son una IA general)?

6. Conclusión

El diseño de prompts parece fácil, pero los usuarios no expertos enfrentan barreras claras: exploran de forma oportunista, se rinden ante fracasos aislados, evitan estrategias efectivas que "se sienten raras" (como dar ejemplos o repetirse) y proyectan expectativas humanas sobre el modelo. Diseñar buenas herramientas de prompting requiere considerar estos sesgos, ofrecer guías, ejemplos y soporte para pruebas sistemáticas, y educar sobre las verdaderas capacidades y límites de los LLMs.

7. Glosario rápido para estudiantes

LLM: Large Language Model. Modelo de lenguaje entrenado con enormes cantidades de texto (ej. GPT-3, ChatGPT).

Prompt: Texto de entrada que se le da al modelo para orientar su respuesta.

Prompt engineering: Práctica de diseñar y refinar prompts para obtener las respuestas deseadas.

Preamble: Parte inicial del prompt que describe el rol y las reglas del bot.

Reminder: Texto breve inyectado justo antes de cada respuesta del bot para reforzar instrucciones.

Regression testing: Volver a probar casos antiguos para asegurar que un cambio nuevo no rompió lo que ya funcionaba.

EUPS: End-User Programming Systems: sistemas que permiten programar a personas sin formación técnica.

iML: Interactive Machine Learning: creación de modelos de ML mediante interacción humana.

CASA: "Computers Are Social Actors": las personas tratan a las computadoras como si fueran actores sociales.

Sobre-generalización: Sacar conclusiones amplias a partir de uno o pocos casos.

Diseño con costuras (seamful): Diseño que hace visibles los límites y la naturaleza técnica del sistema.